DAWL technical research brief

Architecture methodology and mathematical foundations

Prepared by Espen for Yngvar
13 October 2026 · Working research document · Version 1.0

Purpose

DAWL, the Decentralized Asynchronous Weight Ledger, is a working platform for publishing data transformations with evidence that other participants can inspect and replay. The long-term ambition is a jointly maintained, open infrastructure for open-weight models: a ‘Linux of AI’. This brief describes the implemented data and audit foundation, the mathematics behind it, and the experiments that can turn that foundation into a research instrument.

The current system ingests public book texts, creates pinned token snapshots, reconstructs input embeddings on a GPU, records signed lineage, and verifies transformations on independent CPU processes. A configured five-node container swarm adds asynchronous transfer, replication, audit receipts and fault recovery. **No model has been trained in these experiments.**

What the existing evidence supports

Evidence	Observed result	Scope
Platform replay	8 books; 1,438 chunks; 1,657,163 tokens; complete CPU replay passed	Source, tokens and input-vector bytes
Local ingestion	4 CPU workers achieved 2.83 times the 1-worker median throughput	One host; 3 runs per worker count
Swarm recovery	5 nodes converged after injected faults; a false canary approval was contradicted	Configured membership on one host
Sampling experiment	1,803 audits; zero disagreements with the detection oracle	Controlled token corruption; 43 chunks

The central distinction is between **identity** (which bytes were published and by which key), **transformation integrity** (whether replay matches a stated recipe), and **epistemic quality** (whether a future answer is correct or useful). The first two are partially testable here. The third requires separate evaluation; cryptographic provenance does not supply it.

Reading guide: architecture and data flow, pages 2-3; mathematical principles, pages 4-6; recorded experiments, pages 7-9; assurance boundaries, page 10; research use and next experiments, pages 11-12; references and evidence, page 13.

1 Architecture and node responsibilities

A node wears one or more explicitly admitted hats. Producer permissions govern who may publish particular lineage records. Swarm membership governs who may advertise and exchange data or observations. These are separate policies: admitting a provider does not authorize it to rewrite the dataset.

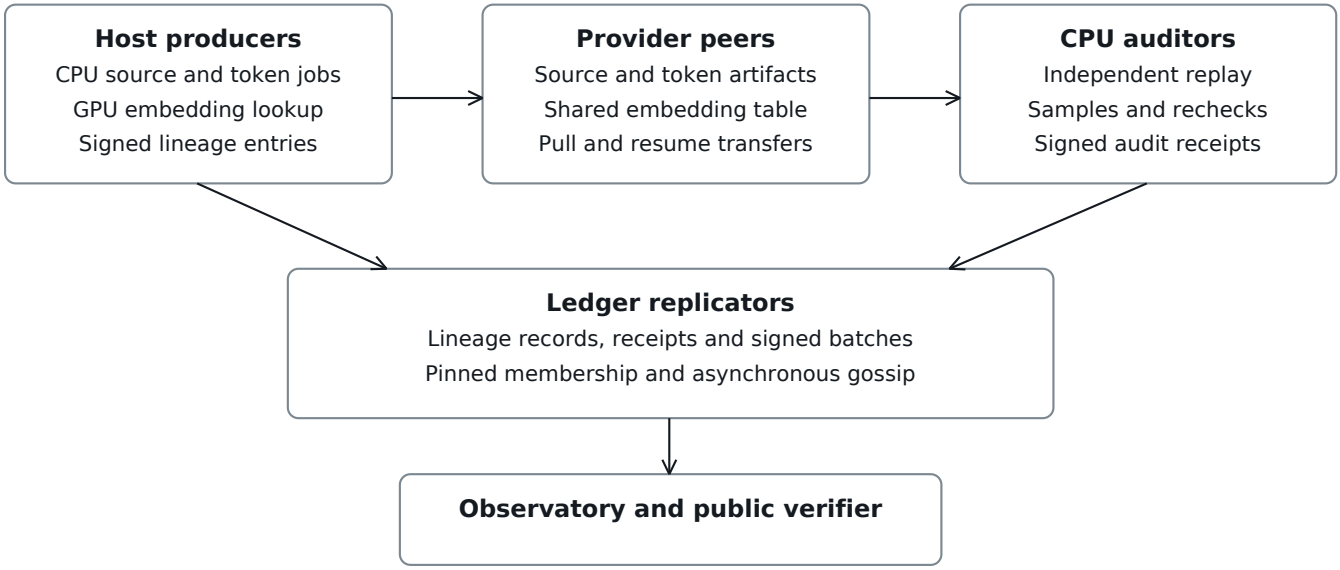


Figure 1. Implemented data and observation flow. Arrows show publication or evidence flow; auditors initiate artifact pulls. The host GPU produced the vector commitments before the swarm exercise. Scheduling new GPU jobs across the swarm remains future work.

Role or component	Responsibility	Current implementation
Host producers	Acquire sources, tokenize, extract and use the pinned table	Local CPU and DirectML GPU paths
PROVIDER 1	Serve content-addressed artifacts and piece inventories	Two provider and replicator containers
AUDITOR 2	Replay data transformations and sign scoped observations	Dedicated CPU auditor; separate dishonest test node
LEDGER_REPLICATOR 4	Validate and exchange lineage and observation sets	Dedicated replicator plus provider replicas
Public verifier	Repeat checks using public policies and artifact bytes	Fresh cache; no private key required

The **data plane** pulls source, token and shared-table objects. The **control plane** exchanges signed membership announcements, inventories, records and receipts. The **observatory** exposes topology, availability, checkpoints and disputes at localhost:8880. It is a view of collected evidence, not an authority that makes a claim true.

Implementation map: src/dawl_node handles source and token snapshots; src/dawl_platform handles lineage, vector commitments and platform replay; src/dawl_swarm handles asynchronous peer state, transfers, jobs and gossip. [E1-E3]

2 Data ingestion and the provenance lifecycle

From a source to a replayable artifact

1. Register the source. A source record pins acquired bytes and descriptive provenance such as origin and declared rights. The executed corpus contains eight Project Gutenberg books: Alice in Wonderland, Pride and Prejudice, Frankenstein, Moby Dick, Great Expectations, Huckleberry Finn, Dracula, and The Adventures of Sherlock Holmes. The registry records exact artifacts, not an abstract promise that any later download is interchangeable.

2. Create immutable chunks and token snapshots. The corpus contains 16 parts and 1,438 UTF-8 chunks under the configured 4,096-byte chunk contract. Tokenization is independent per chunk, uses the pinned real GPT-2 BPE tokenizer, adds no special tokens, and applies no hidden text normalization. Tokenizer bytes and runtime are recorded. Chunk boundaries are part of the recipe because BPE output can depend on those boundaries.

3. Register the reusable embedding dependency. The table extracted from the pinned GPT-2 checkpoint contains 50,257 rows of 768 float32 values. Its byte hash and extraction lineage are published. This establishes the imported artifact’s identity; DAWL did not observe the model’s historical pretraining process.

4. Reconstruct and commit. The GPU gathers the table rows selected by token IDs, hashes each chunk’s resulting vector bytes, and publishes the commitments. Dense token-sequence vector files are not retained or distributed. Providers retain the material from which those vectors can be reconstructed.

5. Verify and record the result. A CPU verifier checks pinned files and lineage, reconstructs source chunks, repeats tokenization and performs NumPy embedding lookup. It compares expected commitments and signs a receipt with the checked scope. Full platform replay and sampled swarm replay are different assurance levels and must remain visible in the interface.

What belongs in a receipt

A useful receipt binds the target vector entry, relevant checkpoint, audit nonce, sampled positions and total chunk count to an outcome, timing and signer. A recheck also identifies the prior claim. Outcomes distinguish **pass**, **invalid**, **unavailable** and unexpected local **error**. An unavailable dependency cannot be reported as an invalid transformation, and a sampled pass cannot be presented as a full-corpus pass.

Bootstrap trust is an explicit input

A verifier must obtain the intended public policy pin independently of the artifacts it is checking. An internally consistent ledger delivered together with an attacker’s replacement policy is not sufficient. Public verification needs no private key, but it still depends on an authentic starting policy, correct verifier software and access to the pinned bytes.

The books provide reproducible engineering inputs, not a financial-advice evaluation set. Recorded source and licence metadata are claims about origins and permissions; hashes do not adjudicate copyright or jurisdiction. The corpus should not be described as universally cleared for every future use. [E1-E2]

3 Commitments signatures and ledger structure

Content identity and authenticated claims

$$\mathrm{id} = \mathrm{SHA256}(C(b)), \qquad \sigma = \mathrm{Sign}_{sk}(D \,\|\, C(b))$$

Here b is a record body, C is the project's deterministic JSON encoding, and D is a signing domain. The implementation sorts keys, uses compact separators, ASCII escapes and rejects non-finite JSON numbers. This is a project encoding contract, not a claim of general JSON canonicalization compatibility. Schema fields identify message types; the shared signing prefix is DAWL/ingestion-node/v2 followed by a zero byte.

SHA-256 provides a practical content identifier under its collision-resistance assumption. Ed25519 authenticates a statement to a public key under the signature scheme's security assumptions. A valid signature does not establish a signer's real-world identity, honesty, execution time or completion of computational work. Key compromise permits new apparently authorized claims. [R1-R2]

Merkle commitments bind ordered chunks

$$h_i = H(D_L \| i \| |x_i| \| x_i), \qquad h_p = H(D_N \| h_l \| h_r)$$

The code domain-separates leaves and internal nodes, commits the leaf index and byte length, and distinguishes source and token domains. Indices use little-endian uint64 and lengths uint32; an unpaired final node is duplicated. An inclusion path requires O(log N) sibling hashes, while constructing the tree requires O(N) hashes. A proof shows inclusion under a particular root; its semantic meaning comes from the signed record and the pinned transformation contract.

The exact binary encoding is specified by src/data_ingestion/protocol.py. This tree is not wire-compatible with Certificate Transparency. RFC 9162 provides useful background on Merkle transparency structures, but DAWL currently provides neither its global log semantics nor its consistency-proof protocol. [R3]

A lineage graph and author chains

Records form a directed acyclic dependency graph: an output names previously admitted parent inputs. Each author additionally has a sequence number and previous-record link. Validation rejects missing parents, sequence gaps and local author forks. Database append-only constraints defend the normal write path; signatures and externally retained checkpoints are needed to detect rewritten or truncated state outside that path.

$$K = H(C(\mathrm{sort}\{\mathrm{id}(e) : e \in L\}))$$

K is a digest of a sorted record set. Equal digests are evidence of equal sets under the hash assumption; they do not impose a total order or establish finality. This checkpoint is not a Merkle root and has no compact consistency proof. Eventual exchange among honest, connected peers with coherent admission and retained data can reconcile sets. This is not Byzantine consensus. The graph's entities, transformations and signers also map conceptually to W3C PROV, without claiming PROV serialization compliance. [R4]

4 Reconstructing vectors and accounting for storage

$$t = T_\theta(x), \quad E \in \mathbb{R}^{V \times d}, \quad X_i = E[t_i, :]$$

T is the pinned tokenizer and chunk recipe; t is a sequence of L token IDs; E is the fixed input embedding table; X is the sequence of selected rows. The executed parameters are V = 50,257 and d = 768. IDs are uint32 little-endian. Vector commitments use row-major, little-endian float32 bytes. Replaying this lookup is exact byte reconstruction, not a statistical approximation.

The same token selects the same table row regardless of surrounding text. These are **input embeddings**, not contextual representations, sentence embeddings, generated answers or learned updates. The CPU and GPU agreement observed here does not imply that arbitrary neural-network training or floating-point reductions are bitwise deterministic.

$$B_{dense} = 4Ld, \quad B_{tokens} = 4L, \quad B_{table} = 4Vd$$

$$B_{reconstruct} = 4L + 4Vd + M$$

M represents the additional published source, metadata, ledger and policy bytes included in the chosen comparison. A table is reusable across corpora that pin the same table identity. Receivers must obtain it at least once or already hold a verified copy; avoiding dense vectors does not eliminate that dependency.

Component	Bytes in the platform run
Raw corpus	5,850,515
Token IDs (L = 1,657,163)	6,628,652
Shared embedding table	154,389,504
Hypothetical retained dense sequence vectors	5,090,804,736
Published package including source, ledger and policy	169,193,602

Token IDs alone use 1/d of the dense representation’s bytes, a 99.8698% reduction. The recorded public package is 96.68% smaller than that dense representation. Neither figure is raw-text compression: token bytes exceed the original text size. The table dominates this small corpus; excluding M, amortization beats dense storage once L exceeds Vd/(d - 1), approximately 50,323 tokens.

These figures exclude development caches, the original full pretrained checkpoint, redundant benchmark stores, software and profiling logs. They measure a specified representation comparison, not total disk savings, network savings or model compression. Corrupt transfers, retries and replication can increase actual network and storage costs. [E2]

5 What a sampled audit can establish

Let N be the number of committed chunks, m the fixed number whose transformation is invalid, and k the number sampled uniformly without replacement. If every selected invalid chunk is detected by a correct replay, the probability of finding at least one invalid chunk is:

$$P_{detect} = 1 - \frac{\binom{N-m}{k}}{\binom{N}{k}}$$

There are binomial(N, k) possible samples. Exactly binomial(N - m, k) contain only good chunks. Subtracting that miss probability from one gives the result. Detection is certain when k exceeds N - m. With one bad chunk, the expression simplifies to k/N.

Population	Sample	Probability of detection
43 chunks; 1 invalid	16 chunks	37.21%
43 chunks; 4 invalid	16 chunks	85.78%
43 chunks; 1 invalid	41 chunks	95.35%
43 chunks; at least 1 invalid	All 43	100% within the replay model

Assumptions are part of the guarantee

The bad set and input commitments must be fixed before an honest unpredictable draw. The sampler must follow the stated distribution; the auditor must recompute correctly; unavailable bytes must remain a separate outcome. If an adversarial auditor chooses easy samples, or knows the challenge before choosing corrupt positions, this calculation is not its security guarantee. A public deterministic seed supports reproducibility but does not by itself supply adversarially safe randomness.

A clean sample therefore means ‘no discrepancy in the checked positions’. It is not a 95% probability that the whole dataset is correct. Such a posterior claim would require an explicit prior and model of corruption. For k small relative to N, the familiar approximation $1 - (1 - m/N)^k$ is useful; the exact finite-population expression is used here.

Books and repeated checks

$$P_{miss,\ stratified} = \prod_{j} \frac{\binom{N_j - m_j}{k_j}}{\binom{N_j}{k_j}}$$

The swarm samples 16 chunks per valid book. This is stratified by book, not uniform over all 1,438 corpus chunks: a chunk in a larger book has a lower selection probability. The product applies to independent draws within books and a fixed corruption allocation. Repeating r genuinely independent fresh draws gives P_{miss}^{r}. Retrying the same job and replaying another node’s same positions add no statistical coverage; they instead test availability or the honesty of that particular claim.

6 R001 validates the sampling mechanism

R001 isolates a failure that signatures alone cannot catch. A miner with a valid signing key changes token content while retaining the source and tokenizer pins, then recomputes token hashes, Merkle roots and signatures. The resulting snapshot is cryptographically coherent but semantically inconsistent with its source. Independent re-tokenization should detect the discrepancy precisely when an altered chunk is sampled. [E4]

The fixture has 43 chunks and 52,991 real BPE tokens. Conditions contain zero, one or four modified chunks; sample sizes are 1, 8, 16 and 43. Each partial-coverage cell has 200 repetitions; each full-coverage cell has one. The pre-specified protocol and fixed laboratory seed produce 1,803 actual audits, with paired sample sequences across corruption conditions.

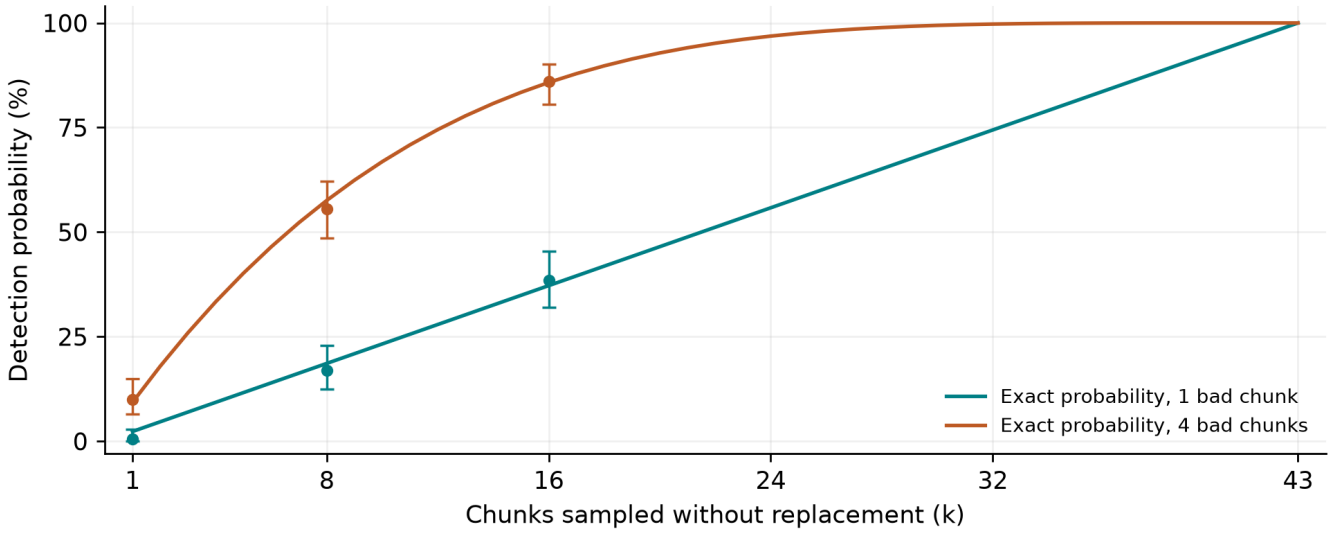


Figure 2. Exact detection curves for N = 43. Points are recorded R001 proportions at k = 1, 8 and 16; bars are 95% Wilson intervals for each 200-trial cell. They show finite Monte Carlo variation, not uncertainty in the combinatorial formula. Full coverage deterministically detects the altered fixtures.

Invalid chunks	k	Detected out of 200	Observed	Exact
1	1 / 8 / 16	1 / 34 / 77	0.5 / 17.0 / 38.5%	2.33 / 18.60 / 37.21%
4	1 / 8 / 16	20 / 111 / 172	10.0 / 55.5 / 86.0%	9.30 / 57.57 / 85.78%

All three fixtures passed complete-file integrity checks. The honest condition produced zero detections in every partial cell. Every trial agreed with the oracle defined by intersection of the sampled and altered positions; oracle mismatches were zero. Full coverage accepted the honest fixture and rejected both altered fixtures.

This validates the implemented source-to-token check for these controlled fixtures and confirms why authentication alone is insufficient. It does not estimate prevalence of real attacks, prove verifier completeness, test a network, or measure human trust. The paired design means cells across corruption conditions must not be treated as independent groups. Timings came from warm, serial execution and are not a performance benchmark.

7 Platform replay and local ingestion scaling

Complete replay of a larger fixed corpus

The platform experiment moves from the 43-chunk fixture to eight locked books. The producer publishes source and token artifacts, the host GPU reconstructs input vectors, and a separate CPU container repeats all source, token and vector checks. All 1,438 chunks passed. The profile records 1,438 DirectML Gather executions; no dense sequence-vector files were written. The 36 signed lineage records converged between producer and auditor stores. [E2]

A fresh public verifier repeated the entire corpus without a private key. Another fresh verifier succeeded against the replica while the original artifact peer was stopped. Container replacement preserved completed receipts and ledger state. In this baseline experiment, selecting the alternate provider was explicit; automatic fallback was exercised in the later swarm.

Different timing scopes must remain separate

Measurement	Recorded duration	Scope
GPU reconstruction and commitments	13.94 seconds	After table and session initialization; includes integrity and hashing
CPU full replay	33.01 seconds	Sum of per-book times; includes first table download

The ratio of these durations is not a CPU versus GPU speedup measurement. A fair comparison would align setup, cache state, transfers, verification work and timing boundaries.

Local parallel ingestion

CPU workers	Median seconds	Observed range	Speedup
1	17.42	17.10 to 17.57	1.00
2	9.80	9.78 to 9.89	1.78
4	6.15	6.09 to 6.35	2.83

Three interleaved runs at each worker count produced 72 book jobs. Every job reproduced its baseline snapshot identity. The timed path includes process startup, tokenization, hashing, signing, file publication and a complete-file consumer check. It excludes GPU reconstruction, HTTP transfer and the separate CPU audit.

These measurements demonstrate useful local parallelism for this corpus, not arbitrary scale. One Windows host and shared disk served all jobs; OS caching and OneDrive effects were uncontrolled. The runs were integration measurements, not a preregistered performance study. The final platform suite had 73 passing tests; after the swarm additions the total was 84. [E2-E3]

8 Asynchronous swarm mechanics and fault recovery

The swarm uses configured bootnodes and signed admission certificates binding a public key, role mask and permitted origin. Announcements have sequences and expiry; persisted high-water marks prevent rollback, and replayed announcements do not refresh liveness. Known members gossip asynchronously. There is no Kademlia DHT, permissionless admission, staking or reward system in this version. [E3]

Providers expose content-addressed objects in 1 MiB pieces. Bounded, pooled HTTP requests, object locks and persisted piece caches support deduplication, fallback and resumption. An untrusted piece catalogue cannot override the complete-object hash pinned by lineage; publication requires final whole-object verification. Audits currently fetch whole source and snapshot files before replaying selected chunks, so sampling does not yet minimize every network transfer.

Auditors keep durable jobs, retain the selected positions on retries, and execute CPU replay separately from gossip. Receipts are signed observations. Replicators group receipt IDs into signed local batches with per-author chains; these are not globally finalized epoch blocks. Incremental gossip cursors avoid repeatedly exchanging the complete observation history.

Five containers and a controlled canary

Two provider/replicators, one auditor, one dedicated replicator and one dishonest auditor ran with separate identities and state, each limited to two CPU cores and 2 GiB RAM. All shared one physical host. The lab copied the 36-record baseline and added a signed, deliberately invalid vector entry. The baseline dataset was unchanged. The dishonest node issued nine dummy approvals, including one for that canary.

Injected condition	Recorded response
Both providers withhold	Unavailable receipts, distinct from invalid transformations
Corruption plus an honest alternative	249 corrupt pieces rejected at the final checkpoint
Provider stops during table download	Partial object withheld; later recovery used 26 piece-cache hits
Replica partition heals	All 5 nodes converge on 37 lineage entries and 40 receipts
Dummy canary approval rechecked	Independent replay contradicts the false claim

The final 40 receipts comprise 9 dummy passes, 16 honest passes, 2 invalid results and 13 unavailable results; there were zero error receipts. All nine original jobs and nine rechecks completed. The eight valid books passed 16 sampled chunks each; the canary failed both its original audit and its recheck. All 79 observations, including 39 signed batches, were independently checked for signatures and batch-chain validity.

Application-level delay, withholding and partitions exercised this implementation’s recovery paths. They are not measurements of real Internet packet loss, independent operators or Internet-scale performance. Existing GPU commitments were inputs; distributed GPU scheduling was not tested.

9 Assurance boundaries and adversarial behaviour

Evidence or mechanism	Supported interpretation	Remaining limitation
Content hash	Bytes match an independently pinned identifier	Does not establish truth, rights or source completeness
Signature and admission	An admitted key authenticated this claim	A key can lie, collude or be compromised
Merkle inclusion proof	A leaf belongs to a committed ordered structure	Does not prove the transformation is correct
Independent replay	Specified checks match at specified positions	Depends on recipe and verifier correctness; samples have limited coverage
Equal set checkpoint	Peers hold the same validated record set	No global consensus, finality or censorship guarantee
Source metadata	A contributor declared an origin or permission	Not legal verification or proof of historical training provenance
Audit timing	A process reported or locally measured a duration	Remote timing is not proof of computational work

The freeloader result is deliberately narrow

A fabricated approval of correct data may agree with honest replay. The experiment catches the dummy approval of the invalid canary because an independent auditor reconstructs the committed vectors and contradicts it. It does not distinguish all honest auditors from freeloaders, prove that a remote node executed a 33-second computation, or establish an incentive-compatible economy. No reward eligibility was granted from the dummy receipts.

Availability is a separate property

A corrupted download can be rejected when the verifier already has the correct content pin. An eclipsed auditor with no honest reachable provider may still be unable to proceed. Treating this as unavailable preserves the distinction between missing evidence and invalid data. Multiple replicas only improve fault independence when their operators, storage and connectivity are meaningfully independent; five containers on one host do not provide that independence.

Limits of the current challenge model

This is deterministic replay with sampled challenges, not a zero-knowledge proof. Selected positions and evidence may be public. A malicious node can fabricate a receipt or copy evidence; policy, independent rechecks and dispute handling determine how such claims are used. Robust adversarial randomness, anti-grinding rules, collusion resistance, admission governance and economic enforcement remain design work.

A rights claim or trusted-looking chain can be misleading even when every byte and signature verifies. Provenance should expose the strength and limits of evidence rather than convert every signed statement into an endorsement. That distinction is also the central question for the proposed human study.

10 Connecting the platform to the trust study

Yngvar's proposed study asks whether independently verifiable provenance changes perceived credibility, reliance and decision quality in an AI advice setting. It also allows the opposite outcome: a technically impressive trail may produce misplaced confidence. DAWL supplies a way to vary access to inspectable evidence while holding the advice itself constant.

Condition	Information presented	Design requirement
A Black box	Advice without provenance information	Same advice and task difficulty across arms
B Declared transparency	Advice plus developer statements about origins and process	Statements must be accurate; no independent verification offered
C Verifiable transparency	Same base statements plus inspectable evidence and a replay route	Distinguish verified facts, declarations and missing evidence

B and C should differ in independent checkability as far as practicable, rather than in the quantity of reassuring prose or a decorative trust badge. The interface should report audit scope and availability plainly. A comprehension pilot can test whether participants understand those distinctions before a powered study. Interface behaviour and evidence access need to be logged without treating clicks as proof of comprehension.

Reliance must be evaluated against advice quality

Expert-checked scenarios can contain both appropriate and intentionally incorrect advice. Trust ratings alone cannot show better decisions. Pre-specify acceptance of good advice, rejection of bad advice, final task accuracy, evidence use and attribution of responsibility. One descriptive measure of discrimination is:

$$\Delta = P(\text{follow} \mid \text{correct}) - P(\text{follow} \mid \text{incorrect})$$

Report the two underlying rates as well: the same difference can mask very different behaviour. A future analysis could estimate condition-by-advice-correctness effects with participant and item variation; the final estimand, power analysis and model should follow Yngvar's study design. This brief reports no human-subject findings and makes no venue or institutional commitment.

The missing bridge to training provenance

Today's evidence concerns ingestion and input embeddings. A study explicitly claiming **training provenance** needs a controlled training or fine-tuning run that binds the selected dataset, tokenizer, parent weights, recipe, software environment and resulting checkpoint. A saved run and hash still do not prove that every declared input was actually used; the research claim must match the strength of replay or other retained evidence.

The platform should remain DAWL's engineering project. A separate research branch can pin the interface and workload used by the study, record deviations, and preserve the original platform direction. No training is required to finish the current ingestion and auditor validation. Nor would later training lineage prove which individual datum caused a particular generated answer.

11 Next experiments and reproducibility discipline

The next engineering step is a controlled systems experiment, with one primary question per experiment and explicit measurement boundaries. The research fork should consume stable platform evidence rather than silently change protocol behaviour during a study. The following are proposed experiments, not completed results.

Experiment	Vary and control	Primary measurements and success condition
Scaling	Corpus size, peer count and cache state; fix hardware and link limits	Verified tokens or bytes per second, tail latency, CPU/RAM, wire bytes, storage amplification; all accepted outputs remain correct
Churn and recovery	Scheduled joins, departures and partitions on independent hosts	Time to resume and converge after healing; no partial object publication or false pass under unavailable evidence
Adversarial replay	Corruption locations, dishonest providers and fabricated receipts	False acceptance and rejection by audit scope; observed detection versus exact sampling expectations
Verifier independence	Different machines and independently maintained replay path	Agreement on canonical artifact bytes and commitments; characterize dependency and platform divergence
Study readiness	Frozen evidence package and controlled provenance interface	Users can distinguish a declaration, sampled check, full replay and unavailable evidence

Record the workload and the environment

Freeze code, dependencies, policy and artifact hashes, resource limits and the fault schedule before each run. Separate cold-cache and warm-cache trials. Interleave conditions, repeat runs and retain failures. Measure verified useful work, separating end-to-end time from active replay and intentional waiting.

The swarm exercise’s eight-valid-books-plus-dispute condition occurred at 344.715 seconds. An intermediate 39-receipt convergence check occurred at 372.733 seconds; the final recheck completed afterward. A separate final snapshot establishes 40-receipt convergence. These observations must not be merged into a single unrecorded final timing. The reusable runner now waits for every recheck before checking convergence. [E3]

Reproduction route

Use docs/platform.md for the baseline and docs/swarm.md for the swarm runbooks. Acquire and verify the pinned large artifacts, load the intended public policies, and then reproduce full or sampled replay with fresh state. A ledger-only export is insufficient for offline reconstruction. Public canary and valid-claim checks already ran with fresh caches and no private keys; the canary verifier’s exit status of 1 is the expected rejection result.

Retain protocols, raw trials, hashes and analysis together. Chunks, jobs, containers and physical machines are not interchangeable independent sample sizes. Yngvar and his team will manage the human study’s ethics, recruitment and analysis protocol.

12 Evidence map and references

This document describes the implementation reviewed at commit ef9fff50ec07f1e1a47900e7a905de7678d5f1bd on 13 October 2026. Historical measurements retain their own recorded code fingerprints; this later revision is not retroactively assigned to every trial. Paths below are relative to the relevant repository unless noted.

Project evidence

E1 Architecture and code. docs/platform.md; docs/swarm.md; src/dawl_node; src/dawl_platform; src/dawl_swarm. Merkle encoding: src/data_ingestion/protocol.py. Sampling mathematics: src/data_ingestion/validator_node.py.

E2 Platform execution. docs/platform_run_001.md and docs/results/platform-run-001/. summary.json indexes the archived measurements; benchmark.json, ledger.jsonl, trust.json, audit-receipts.json, public-verification.json and deployment.json retain supporting observations. Capture commit: 211b21a39b60fe540c61024888c2497c62bc8a09. Some earlier implementation fingerprints are not archived Git revisions.

E3 Swarm execution. docs/swarm_run_001.md and docs/results/swarm-run-001/. report.json retains exercise timings; final-deployment.json retains the final 40-receipt state; observations.jsonl and observation-verification.json retain signed observations and validation. Public replay reports, phase snapshots, runtime versions, tests and development-failure logs are included. artifact-index.json records file hashes. Final suite: 84 passing tests; UI API and JavaScript checks passed, visual browser review was not performed.

E4 R001 research experiment. Separate research worktree, branch codex/verifiable-provenance-research: research/experiments/R001-protocol.md and artifacts/research/R001-run-001/{summary.json,trials.jsonl}. Source commit: dda65b25a42fee523d61745f0d0acb9eca13438a. Run completed 13 October 2026. A shareable evidence copy and file checksums accompany this PDF; large source/model artifacts and private keys are excluded.

Selected reproducibility pins

Baseline dataset entry

`d912bf12b89bbb811cd768293d8ad301c4d6fbe4b16de2af9910547612a8ded5`

Baseline trust policy SHA256

`b859f160c0b3c79b710f6674c8e8110f5276de85ac97a7e4319a8a9d3094c91f`

Swarm policy SHA256

`80de79ff0a30ac6cd73138b9e7333efb88a8d208e58476646a69ddd98c05febe`

External foundations

R1 NIST. *Secure Hash Standard*, FIPS 180-4. csrc.nist.gov/pubs/fips/180-4/upd1/final.

R2 Josefsson, S. and Liusvaara, I. (2017). *Edwards-Curve Digital Signature Algorithm (EdDSA)*, RFC 8032. rfc-editor.org/rfc/rfc8032.

R3 Laurie, B. et al. (2021). *Certificate Transparency Version 2.0*, RFC 9162. rfc-editor.org/rfc/rfc9162.

R4 W3C (2013). *PROV-DM: The PROV Data Model*. w3.org/TR/prov-dm/.

The hypergeometric detection formula is derived on page 6. The GPT-2 artifact is pinned to openai-community/gpt2 revision 607a30d783dfa663caf39e06633721c8d4cfcd7e; its upstream model card and extraction provenance are retained with the platform assets. This brief is an engineering handoff, not a systematic literature review or a claim of novelty.